

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin Lanjut]]

[!cornell] Vanishing Gradient Problem in RNNs

## Questions/Cues

- Mengapa vanishing gradient terjadi pada RNN?
- Bagaimana fungsi aktivasi memengaruhi propagasi gradien?
- Apa dampak eksponensial perkalian gradien?
- Solusi alternatif selain LSTM untuk masalah ini
- Perbedaan vanishing vs exploding gradient

## Reference Points

- Lecture\_IF3270\_RNN.pptx (Slides 6, 8)
- Goodfellow et al. Deep Learning (Chapter 10)
- Raschka Machine Learning with PyTorch (Chapter 15)

## Pengertian Vanishing Gradient Problem

Vanishing Gradient Problem adalah fenomena dalam pelatihan **jaringan saraf berulang (RNN)** di mana gradien error menjadi semakin kecil saat propagasi mundur melalui lapisan waktu. Hal ini menyebabkan: - **Bobot jaringan tidak terupdate** secara signifikan pada lapisan awal - **Pembelajaran jangka panjang (long-term dependencies)** menjadi tidak mungkin - Konvergensi model melambat atau berhenti sama sekali Masalah ini terutama kritis pada RNN karena sifatnya yang **berulang (recurrent)**. Setiap langkah waktu setara dengan lapisan tambahan dalam jaringan feedforward yang sangat dalam. Contoh: RNN dengan 100 langkah waktu setara dengan jaringan feedforward 100 lapisan.

## Penyebab Matematis

Penyebab utama vanishing gradient terletak pada **aturan rantai (chain rule)** dalam kalkulus diferensial. Dalam RNN, gradien error  $\partial E / \partial W$  dihitung dengan mengalikan serangkaian turunan parsial:  $\partial E / \partial W = \partial E / \partial h_t * \partial h_t / \partial h_{t-1} * \partial h_{t-1} / \partial h_{t-2} * \dots * \partial h_1 / \partial W$  Jika setiap elemen  $\partial h_t / \partial h_{t-1}$  bernilai  $< 1$

(karena fungsi aktivasi tanh/sigmoid yang memiliki turunan maksimal 1), hasil perkalian akan **mengecil secara eksponensial** seiring bertambahnya langkah waktu. Contoh numerik: jika rata-rata turunan parsial = 0.8, setelah 50 langkah waktu gradien akan menjadi  $0.8^{50} \approx 1.4e-05$  (nyaris nol).

### Dampak pada Pelatihan Model

1. **Gagal mempelajari ketergantungan jangka panjang:** Model hanya mampu mengingat pola dalam rentang waktu pendek (misal 5-10 langkah)
2. **Stagnasi parameter:** Bobot di lapisan awal tetap statis meski pelatihan berlanjut
3. **Bias terhadap informasi terbaru:** Model menjadi terlalu fokus pada input terkini dan mengabaikan konteks historis Contoh nyata: Pada analisis sentimen kalimat panjang, model mungkin hanya mempertimbangkan kata terakhir ("Saya suka film ini tapi aktingnya buruk, alur ceritanya kacau, dan endingnya tidak memuaskan → positif?")

### Solusi Umum (Non-LSTM)

1. **Fungsi Aktivasi ReLU:** Memiliki turunan konstan 1 untuk input positif, mencegah perkalian gradien mengecil. Namun rentan terhadap exploding gradient.
2. **Inisialisasi Bobot Ortogonal:** Mempertahankan norma gradien dengan menginisialisasi matriks bobot recurrent sebagai matriks ortogonal ( $W^T W = I$ )
3. **Gradient Clipping:** Membatasi nilai maksimum gradien selama backpropagation:  

```
python grad_norm = np.linalg.norm(gradients) if grad_norm > threshold:
    gradients *= threshold / grad_norm
```
4. **Skip Connections:** Menambahkan jalur langsung yang melewati beberapa lapisan waktu (residual connections), memungkinkan gradien mengalir tanpa melalui fungsi aktivasi.

[!cornell] ##### Summary

**Vanishing gradient** pada RNN terjadi karena **perkalian eksponensial gradien kecil** melalui banyak langkah waktu, terutama saat menggunakan fungsi aktivasi seperti tanh/sigmoid. Masalah ini **menghambat pembelajaran ketergantungan jangka panjang** dan menyebabkan stagnasi parameter. Solusi alternatif termasuk penggunaan **ReLU, inisialisasi ortogonal, gradient clipping, dan arsitektur residual**, sementara **LSTM** (tidak dibahas detail di sini) merupakan solusi khusus melalui mekanisme gating.

[!ad-libitum]- Additional Information

## Analisis Matematis Mendalam

Untuk RNN sederhana dengan fungsi aktivasi tanh, turunan parsial  $\partial h_t / \partial h_{t-1} = W * (1 - \tanh^2(a_{t-1}))$ . Karena  $0 < (1 - \tanh^2(x)) \leq 1$  untuk semua  $x$  riil, perkalian berulang akan menyebabkan gradien menyusut secara eksponensial jika nilai eigen  $W < 1$ . Analisis nilai eigen matriks bobot menunjukkan bahwa vanishing gradient terjadi ketika radius spektral  $\rho(W) < 1$ .

## Truncated BPTT

Teknik **Truncated Backpropagation Through Time** membatasi panjang unrolled network selama pelatihan: - Membagi urutan panjang menjadi segmen pendek (biasanya 50-100 langkah) - Mengupdate bobot berdasarkan gradien lokal setiap segmen - Mengurangi komputasi dan memitigasi vanishing gradient dengan membatasi kedalaman efektif

## Perbandingan Exploding Gradient

| Karakteristik | Vanishing Gradient      | Exploding Gradient           |
|---------------|-------------------------|------------------------------|
| Penyebab      | $\rho(W) < 1$           | $\rho(W) > 1$                |
| Gejala        | Gradien $\rightarrow 0$ | Gradien $\rightarrow \infty$ |
| Solusi        | ReLU, Clipping          | Gradient Clipping            |

## Studi Kasus Numerik

Simulasi perkalian gradien untuk RNN 10 langkah dengan  $W = 0.9$  dan turunan aktivasi rata-rata 0.6: Langkah 1:  $\text{grad} = 0.9 * 0.6 = 0.54$  Langkah 5:  $\text{grad} \approx (0.54)^5 = 0.045$  Langkah 10:  $\text{grad} \approx (0.54)^{10} = 0.0021$

## Self-Exploration Projects

1. Implementasikan RNN sederhana untuk sequence classification pada dataset IMDB. Amati perbedaan akurasi ketika menggunakan aktivasi tanh vs ReLU.
2. Visualisasi norma gradien selama pelatihan dengan TensorBoard untuk mengamati fenomena vanishing gradient.

## Tools dan Resources

- PyTorch: `torch.nn.utils.clip_grad_norm` untuk gradient clipping
- TensorFlow: `tf.clip_by_global_norm` untuk implementasi clipping optimal

- Paper: "On the difficulty of training recurrent neural networks" (Pascanu et al., 2013)

### Further Reading

- "Understanding the exploding gradient problem" oleh Pascanu et al. (2012)
- Bab 10 "Sequence Modeling: Recurrent and Recursive Nets" dari Deep Learning (Goodfellow et al.)
- Tutorial interaktif: <https://distill.pub/2019/memorization-in-rnns/>

---

type: Note cssclasses: - cornell-notes

---

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] LSTM Architecture: Components and Gates

### Questions/Cues

- Mengapa LSTM memiliki cell state tambahan?
- Bagaimana gerbang lupa mengontrol informasi?
- Peran gerbang input dalam update state?
- Bagaimana gerbang output menghasilkan hidden state?
- Apa perbedaan struktural RNN vs LSTM?

### Reference Points

- IF3270 Slides (Halaman 8-15, 21-22)
- <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

### Konsep Dasar Arsitektur LSTM

Long Short-Term Memory (LSTM) adalah jenis khusus Recurrent Neural Network (RNN) yang dirancang untuk mengatasi keterbatasan memori jangka panjang pada RNN konvensional. **Cell state** ( $C_t$ ) berfungsi sebagai "memori jangka panjang" yang mengalir melalui seluruh rantai pemrosesan, mempertahankan informasi relevan dari langkah sebelumnya. Berbeda dengan RNN biasa yang hanya memiliki hidden state ( $h_t$ ), LSTM memperkenalkan tiga gerbang (gate) yang mengatur aliran informasi: Forget Gate, Input Gate, dan Output Gate. Arsitektur LSTM terdiri dari

empat lapisan neural network yang saling berinteraksi dalam setiap modul berulang. Setiap gerbang menggunakan fungsi aktivasi sigmoid (menghasilkan nilai 0-1) untuk menentukan proporsi informasi yang dilewatkan. Sistem gerbang ini memungkinkan LSTM secara selektif mengingat atau melupakan informasi dari input baru dan state sebelumnya.

## Mekanisme Forget Gate

Forget Gate bertanggung jawab untuk menentukan informasi mana yang akan dipertahankan atau dibuang dari cell state sebelumnya. Gerbang ini menerima input dari hidden state sebelumnya ( $h_{t-1}$ ) dan input saat ini ( $x_t$ ), kemudian menghasilkan vektor nilai antara 0 dan 1 melalui fungsi sigmoid:  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ . Nilai 1 menunjukkan "pertahankan seluruh informasi" sementara 0 berarti "buang seluruh informasi". Misalnya, dalam analisis sentimen kalimat, forget gate dapat memutuskan untuk membuang informasi tentang subjek kalimat sebelumnya jika konteks sudah berubah.

## Operasi Input Gate dan Kandidat Update

Input Gate mengatur penambahan informasi baru ke cell state melalui dua komponen: 1. **Lapisan sigmoid** memutuskan nilai mana yang akan diupdate 2. **Lapisan tanh** membuat vektor nilai kandidat baru ( $\tilde{C}_t$ )  $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$   $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$ . Kombinasi kedua komponen ini menentukan seberapa besar informasi baru akan mempengaruhi cell state. Analoginya seperti editor yang memutuskan bagian mana dari draf baru yang layak dimasukkan ke dokumen final.

## Update Cell State

Proses update cell state menggabungkan operasi dari forget gate dan input gate:  $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$ . Perkalian antara  $f_t$  dan  $C_{t-1}$  memfilter informasi lama yang tidak relevan, sementara penambahan  $i_t * \tilde{C}_t$  memasukkan informasi baru yang penting. Proses linear ini memungkinkan transfer informasi yang stabil tanpa degradasi gradien.

## Output Gate dan Hidden State

Output Gate mengontrol informasi yang akan ditampilkan pada hidden state ( $h_t$ ) berdasarkan cell state saat ini:  $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$   $h_t = o_t * \tanh(C_t)$ . Hidden state berfungsi sebagai "memori jangka pendek" yang digunakan untuk prediksi dan diteruskan ke langkah berikutnya. Proses filtering melalui tanh memastikan nilai output berada dalam rentang  $[-1, 1]$ . Contoh aplikasi: dalam

prediksi urutan teks,  $h_t$  menentukan probabilitas karakter berikutnya berdasarkan konteks saat ini.

[!cornell] ##### Summary

**LSTM** memperkenalkan **cell state** sebagai memori jangka panjang yang diatur oleh tiga gerbang spesialis: **Forget Gate** bertugas menyaring informasi tidak relevan, **Input Gate** mengontrol penambahan informasi baru, dan **Output Gate** menentukan representasi hidden state. **Model empat lapisan interaktif** ini memungkinkan retensi informasi jarak jauh yang lebih baik dibanding RNN konvensional. Implementasi parameter menggunakan formula  $(m+n+1)*4*n + (n+1)*k$  dimana  $m$  dimensi input,  $n$  neuron tersembunyi, dan  $k$  dimensi output.

[!ad-libitum]- Additional Information

## Evolusi Arsitektur dari LSTM ke ReGU

Perkembangan LSTM melahirkan varian seperti **GRU (Gated Recurrent Unit)** yang menggabungkan cell state dan hidden state serta mengurangi gerbang menjadi dua. Pada tahun 2019, **Residual Gated Unit (ReGU)** memperkenalkan koneksi residual (shortcut connection) yang memungkinkan aliran gradien lebih lancar melalui jaringan yang sangat dalam. Arsitektur ReGU mengadopsi mekanisme gate dari GRU dengan tambahan jalur paralel yang menghubungkan input langsung ke output.

## Optimasi Parameter pada Implementasi

Jumlah parameter LSTM dapat dihitung dengan rumus  $4(n(m+n+1)) + k(n+1)$  dimana: -  $n$ : jumlah unit LSTM -  $m$ : dimensi input -  $k$ : dimensi output Parameter ini mencakup empat set bobot ( $W_f$ ,  $W_i$ ,  $W_c$ ,  $W_o$ ) untuk gerbang dan cell state. Pada implementasi Keras, parameter `input_shape` mendefinisikan panjang urutan (time steps) dan fitur input per time step.

## Studi Kasus: Prediksi Deret Waktu

Untuk implementasi masalah many-to-one (misal: prediksi harga saham), model Keras biasanya menggunakan lapisan LSTM dengan 10-100 neuron, diikuti dense layer dengan aktivasi linear untuk output regresi. Critical hyperparameter meliputi jumlah time steps (jendela observasi) dan ukuran batch yang mempengaruhi kemampuan model menangkap pola temporal jangka panjang.

## Proyek Eksplorasi Mandiri

1. Implementasikan LSTM untuk prediksi cuaca dengan dataset historis: Eksperimen dengan variasi time steps (3 hari vs 7 hari) dan ukuran hidden layer

2. Bangun model klasifikasi sentimen Twitter dengan embedding layer + LSTM: Bandingkan akurasi antara arsitektur LSTM dan GRU
3. Visualisasi cell state dan gerbang: Gunakan library LSTMVis untuk melacak bagaimana informasi dipertahankan/diubah sepanjang urutan teks

### Sumber Lanjutan

- "Deep Learning" (Goodfellow et al.) Bab 10: Sequence Modeling
- Paper asli: Hochreiter & Schmidhuber (1997) "Long Short-Term Memory"
- Tutorial interaktif: <https://lstm.seas.harvard.edu>
- Dokumentasi TensorFlow: <https://www.tensorflow.org/guide/keras/rnn>

---

type: Note cssclasses: - cornell-notes

---

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Forward Propagation in LSTM Networks

### Questions/Cues

- Bagaimana aliran informasi dalam forward propagation LSTM?
- Peran fungsi aktivasi sigmoid dan tanh dalam LSTM?
- Tahapan update cell state pada forward pass?
- Implementasi forward propagation di Keras?
- Perhitungan parameter pada jaringan LSTM?

### Reference Points

- Lecture\_IF3270.pptx (Slides 8-15, 19-22)
- Medium\_Article\_LSTM\_Implementation (Pages 17-19)

### Arsitektur Dasar LSTM

Long Short-Term Memory (LSTM) merupakan jenis khusus Recurrent Neural Network (RNN) yang dirancang untuk menangani ketergantungan jangka panjang. Modul berulang pada LSTM terdiri dari beberapa lapisan yang saling berinteraksi, berbeda dengan RNN standar yang hanya menggunakan lapisan tanh tunggal. Selama fase forward propagation, informasi mengalir melalui tiga tahap utama:

komputasi gate, update cell state, dan generasi output. Contoh analogi: Bayangkan LSTM seperti sistem penyaringan surat di kantor pos. Setiap gate bertindak sebagai petugas yang memutuskan surat mana yang disimpan (input gate), dibuang (forget gate), atau dikirim ke tujuan (output gate). Cell state berperan sebagai ban berjalan yang membawa semua informasi penting antar tahap pemrosesan.

## Tahapan Forward Propagation

**1. Komputasi Gate:**

- **Forget Gate:** Menghitung informasi yang akan dipertahankan dari cell state sebelumnya menggunakan fungsi sigmoid. Nilai mendekati 0 menunjukkan informasi akan dibuang, sedangkan nilai mendekati 1 dipertahankan.
- **Input Gate:** Menentukan nilai update baru menggunakan fungsi sigmoid dan tanh. Sigmoid memfilter informasi penting, sedangkan tanh menghasilkan vektor kandidat update.

**2. Update Cell State:** Cell state diperbarui dengan menggabungkan informasi dari forget gate dan input gate:  $C_t = (C_{t-1} * f_t) + (i_t * \tilde{C}_t)$  di mana  $f_t$  adalah output forget gate,  $i_t$  output input gate, dan  $\tilde{C}_t$  vektor kandidat baru.

**3. Generasi Output:** Output gate mengatur informasi yang akan dikirim ke hidden state berikutnya. Cell state yang telah diperbarui diproses melalui tanh kemudian dikalikan dengan output gate:  $h_t = o_t * \tanh(C_t)$

## Contoh Implementasi Praktis

Implementasi forward propagation LSTM menggunakan Keras untuk model Many-to-One:

```
python from keras.models import Sequential from keras.layers import LSTM, Dense
model = Sequential()
model.add(LSTM(10, input_shape=(50,1))) # 10 neuron, input 50 timestep
model.add(Dense(1, activation='linear'))
```

Pada contoh ini, lapisan LSTM memproses urutan data 50 langkah waktu dengan 1 fitur per langkah. Output gate terhubung ke lapisan Dense untuk menghasilkan prediksi numerik.

## Visualisasi Aliran Data

Diagram alur forward propagation: Input ( $x_t$ ) → [Gate Layer] → [Cell State Update] → [Output Gate] → Hidden State ( $h_t$ ) ↑ ↑ ↑ Hidden State ( $h_{t-1}$ ) Cell State ( $C_{t-1}$ ) Cell State ( $C_t$ )

Setiap panah merah menunjukkan aliran data selama forward pass, dengan operasi element-wise multiplication (\*) dan addition (+).

[!cornell] ##### Summary

**Forward propagation pada LSTM** melibatkan tiga tahap inti: komputasi gate menggunakan fungsi sigmoid dan tanh, update cell state melalui kombinasi informasi lama dan baru, serta generasi output melalui filter output gate. Proses ini memungkinkan LSTM **mempertahankan informasi relevan** antar langkah waktu



dengan mekanisme gate yang terkelola. Implementasi praktis menggunakan framework seperti Keras menunjukkan konfigurasi parameter `input_shape` yang menentukan dimensi urutan waktu. **Perhitungan parameter** LSTM secara signifikan lebih kompleks dibanding RNN standar karena adanya empat transformasi linear berbeda dalam setiap sel.

[!ad-libitum]- Additional Information

## Perhitungan Parameter LSTM

Rumus umum perhitungan parameter untuk LSTM dengan: -  $m$ : dimensi input -  $n$ : jumlah neuron LSTM -  $k$ : dimensi output

$$\text{Total parameter} = 4 \cdot (m \cdot n + n^2 + n) + (n \cdot k + k)$$

Contoh untuk arsitektur dengan 10 neuron, input 1 dimensi, output 1 dimensi:  $= 4 \cdot (1 \cdot 10 + 10^2 + 10) + (10 \cdot 1 + 1) = 4 \cdot (10 + 100 + 10) + 11 = 4 \cdot 120 + 11 = 491$  parameter. Komponen utama berasal dari empat set weight matrices (input gate, forget gate, output gate, dan cell state candidate) serta bias untuk masing-masing gate.

## Variasi Arsitektur LSTM Modern

**1. Peephole Connections:** Memungkinkan gate mengakses cell state secara langsung tanpa melalui hidden state. Memodifikasi persamaan gate menjadi:  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + U_f \cdot C_{t-1} + b_f)$

**2. Convolutional LSTM:** Mengganti transformasi linear dengan operasi konvolusi, cocok untuk data spasio-temporal seperti video. Digunakan secara luas dalam video captioning (referensi terlarang - hanya disebutkan tanpa penjelasan).

## Optimasi Implementasi

Teknik optimasi komputasi forward propagation: - **Kernel Fusion:** Menggabungkan operasi matriks untuk mengurangi overhead memori - **Quantization:** Menggunakan presisi 16-bit untuk komputasi - **Parallelization:** Memproses multiple time steps secara paralel pada GPU

## Self-Exploration Projects

1. Implementasikan forward propagation LSTM manual dengan NumPy untuk input sequence [0.5, 1.2, -0.3] dan visualisasikan perubahan cell state tiap langkah waktu
2. Bandingkan output LSTM Keras dengan implementasi manual menggunakan bobot yang sama

3. Eksperimen dengan berbagai fungsi aktivasi alternatif untuk gate (contoh: sigmoid vs hard sigmoid)

### Tools dan Referensi

- **Library:** Keras, PyTorch Lightning, CuDNN-LSTM
- **Visualisasi:** TensorBoard LSTM Traces, Netron
- **Buku:** "Deep Learning" (Goodfellow et al.) Bab 10 - Sequence Modeling
- **Paper:** "LSTM: A Search Space Odyssey" (Greff et al., 2017)

type: Note cssclasses: - cornell-notes

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Backpropagation Through Time (BPTT) in RNNs

## Questions/Cues

- Mengapa BPTT diperlukan untuk RNN?
- Bagaimana tahapan forward pass BPTT?
- Bagaimana menghitung gradien error mundur waktu?
- Apa perbedaan BPTT penuh dan terpotong?
- Bagaimana BPTT pada jaringan bidirectional?

## Reference Points

- Materi Kuliah IF3270 (Slides 23-36)
- Goodfellow et al. Deep Learning (Chapter 10)
- Raschka et al. Machine Learning with Pytorch and Scikit-Learn (Chapter 15)

### Konsep Dasar BPTT

Backpropagation Through Time (BPTT) merupakan algoritma pembelajaran untuk Jaringan Saraf Berulang (RNN) yang memperluas backpropagation standar dengan memperhitungkan ketergantungan temporal. Berbeda dengan jaringan feedforward yang hanya memiliki jalur maju tunggal, RNN memiliki siklus yang membuat perhitungan gradien harus dilakukan secara berurutan untuk setiap langkah waktu. BPTT bekerja dengan "membentangkan" RNN menjadi jaringan feedforward virtual

di sepanjang sumbu waktu. Setiap langkah waktu dianggap sebagai lapisan tersendiri dengan parameter yang sama. Contoh sederhana: untuk kalimat 3 kata, RNN akan dibentangkan menjadi 3 lapisan dengan bobot identik di setiap lapisan. Pendekatan ini memungkinkan perhitungan gradien yang akurat melalui rantai aturan turunan berantai.

## Proses Forward Pass

Pada fase forward pass, RNN memproses data sekuensial langkah demi langkah sambil menyimpan nilai aktivasi (hidden state) setiap langkah waktu:

1. **Inisialisasi:** Hidden state awal ( $h_0$ ) biasanya diinisialisasi dengan nilai nol atau acak kecil
2. **Propagasi:** Untuk setiap langkah waktu  $t$ :
  - Hitung hidden state baru:  $h_t = \sigma(W_{xh} * x_t + W_{hh} * h_{t-1} + b_h)$
  - Hitung output:  $y_t = \sigma(W_{hy} * h_t + b_y)$
3. **Penyimpanan:** Simpan semua nilai  $h_t$  dan  $y_t$  untuk digunakan dalam backward pass

Contoh numerik: Untuk input  $[0.5, 1.25]$ , pertama hitung  $h_1$  menggunakan  $x_1=0.5$ , lalu  $h_2$  menggunakan  $x_2=1.25$  dan  $h_1$ . Nilai  $h$  ini akan digunakan dalam perhitungan gradien mundur.

## Proses Backward Pass

Fase backward pass menghitung gradien error secara berurutan dari langkah waktu terakhir ke awal:

1. **Hitung error output:**  $\delta_y(t) = (y_{pred}(t) - y_{true}(t)) * f'(y(t))$
2. **Propagasi mundur:** Untuk  $t = T$  sampai  $t = 0$ :
  - Hitung gradien untuk parameter berdasarkan rantai turunan
  - Update hidden state error:  $\delta_h(t) = (W_{hy}^T \delta_y(t) + W_{hh}^T \delta_h(t+1)) * f'(h(t))$
3. **Akumulasi gradien:** Total gradien adalah penjumlahan gradien dari semua langkah waktu

Diferensial dilakukan menggunakan aturan rantai melalui seluruh jalur temporal, yang membuat komputasi menjadi intensif untuk urutan panjang.

## Truncated BPTT

Untuk mengatasi kompleksitas komputasi BPTT penuh, digunakan Truncated BPTT yang membatasi panjang urutan:

- **Mekanisme:** Hanya melakukan backpropagation untuk  $\tau$  langkah terakhir ( $\tau \ll$  panjang urutan sebenarnya)
- **Keuntungan:** Menghemat memori dan komputasi
- **Kekurangan:** Gradien mungkin tidak mencapai awal urutan panjang
- **Implementasi:** Pada praktiknya,  $\tau$  biasanya antara 50-200 langkah tergantung aplikasi

Contoh implementasi: Dalam pemrosesan teks buku, kita mungkin memotong menjadi segmen 100 kata untuk setiap iterasi update.

## BPTT pada Bidirectional RNN

Pada arsitektur bidirectional, BPTT dijalankan untuk kedua arah (maju dan mundur):

1. **Forward pass ganda:** Hitung hidden state untuk kedua arah ( $\rightarrow$  dan  $\leftarrow$ )
2. **Backward pass terpisah:** Hitung gradien secara independen untuk setiap arah
3. **Kombinasi gradien:** Gabungkan gradien dari kedua arah sebelum update parameter

Tantangan utama adalah sinkronisasi antara kedua aliran informasi dan manajemen memori yang lebih kompleks.

[!cornell] ##### Summary

**Backpropagation Through Time (BPTT)** adalah algoritma fundamental untuk pelatihan RNN yang memperluas prinsip backpropagation dengan memperhitungkan ketergantungan temporal melalui pembentangan jaringan sepanjang sumbu waktu. Proses ini melibatkan **fase forward pass** untuk menghitung output dan menyimpan aktivasi setiap langkah waktu, diikuti **fase backward pass** yang menghitung gradien secara berurutan dari akhir ke awal urutan. Implementasi praktis sering menggunakan **Truncated BPTT** untuk menangani urutan panjang dengan membatasi rentang propagasi gradien. Pada arsitektur bidirectional, BPTT dijalankan untuk kedua arah dengan mekanisme kombinasi gradien yang hati-hati.

[!ad-libitum]- Additional Information

## Derivasi Matematis Lengkap

Turunan lengkap untuk parameter RNN dengan fungsi aktivasi tanh:

Misalkan fungsi error  $E = \sum E_t$ , dengan  $E_t = \frac{1}{2}(y_t - \hat{y}_t)^2$

Turunan terhadap  $W_{hh}$ :  $\frac{\partial E}{\partial W_{hh}} = \sum_{t=1}^T [\sum_{k=t}^T (\frac{\partial E_k}{\partial h_t}) \cdot \frac{\partial h_t}{\partial W_{hh}}]$

$\frac{\partial h_t}{\partial W_{hh}} = \text{diag}(\tanh'(W_{xh} x_t + W_{hh} h_{t-1})) \cdot (h_{t-1} + W_{hh} \frac{\partial h_{t-1}}{\partial W_{hh}})$

Relasi rekursif ini menunjukkan sifat komputasi yang eksponensial dalam BPTT penuh.

## Optimasi Memori dengan Gradient Checkpointing

Teknik untuk mengurangi penggunaan memori dalam BPTT:

- **Prinsip:** Hanya menyimpan subset hidden state selama forward pass
- **Re-komputasi:** Menghitung ulang state yang tidak disimpan saat backward pass
- **Trade-off:** Mengurangi memori  $O(\sqrt{T})$  dengan komputasi tambahan  $O(T)$

Implementasi umum dalam framework seperti PyTorch menggunakan `torch.utils.checkpoint`.

## Studi Kasus: Implementasi Numerik

Contoh update parameter untuk input sederhana: `` Inisialisasi:  $W = 0.5$  Forward pass:  $t=0$ :  $h_0 = \tanh(W * 0.5) = 0.462$   $t=1$ :  $h_1 = \tanh(W * 0.462 + 1.25) = 0.843$  Error:  $E = 0.5 * (0.843 - 1.25)^2 = 0.0827$

Backward pass:  $\partial E / \partial W = \partial E / \partial h_1 * [\partial h_1 / \partial W + \partial h_1 / \partial h_0 * \partial h_0 / \partial W] = (0.843 - 1.25) * [0.462(1 - 0.843^2) + 0.5(1 - 0.462^2) * (1 - 0.843^2)] = (-0.407) * [0.183 + 0.193] = -0.153$

Update ( $\eta=0.1$ ):  $W_{\text{new}} = 0.5 - 0.1 * (-0.153) = 0.5153$  ``

## Aplikasi dalam Pemrosesan Bahasa Alami

Implementasi BPTT untuk tugas prediksi kata: - **Dataset**: Kumpulan artikel berita (urutan 10.000 kata) - **Arsitektur**: RNN dengan 128 unit tersembunyi - **Truncation**:  $\tau = 64$  langkah - **Teknik**: Gradient clipping pada 1.0 untuk mencegah ledakan gradien - **Hasil**: Akurasi validasi 58.7% pada prediksi kata berikutnya

## Bibliografi Lanjutan

1. "Backpropagation Through Time: What It Does and How to Do It" - Werbos (1990)
2. "On the difficulty of training recurrent neural networks" - Pascanu et al. (2013)
3. "Truncated Backpropagation Through Time" - Mikolov et al. (2014)
4. Dokumentasi TensorFlow: Custom Training Loops for RNNs

## Projek Eksplorasi Mandiri

1. Implementasikan BPTT dari awal untuk model RNN sederhana menggunakan NumPy
2. Bandingkan kinerja BPTT penuh dan truncated pada dataset sinyal waktu (contoh: prediksi saham)
3. Eksperimen dengan teknik optimasi memori seperti gradient checkpointing pada model besar

## Alat dan Sumber Daya

- **Perpustakaan**: TensorFlow/PyTorch RNN modules, CuDNN-optimized LSTM
- **Dataset**: Penn Treebank, Google Billion Words
- **Visualisasi**: TensorBoard Trace View untuk profiling komputasi BPTT

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Encoder-Decoder Models in Sequence Learning

## Questions/Cues

- Apa tujuan model sequence-to-sequence?
- Bagaimana encoder mengolah input sequential?
- Fungsi apa yang dimiliki vector encoder?
- Mengapa decoder memerlukan hidden state awal?
- Bagaimana aplikasi praktis encoder-decoder?

## Reference Points

- Lecture\_IF3270.pptx (Slides 37-43)
- Goodfellow et al. Deep Learning (Chapter 10)

## Konsep Dasar Sequence-to-Sequence

Model sequence-to-sequence (seq2seq) dirancang untuk memetakan input berurutan dengan panjang tetap ke output berurutan yang mungkin memiliki panjang berbeda. Pendekatan ini menjadi dasar untuk berbagai tugas pemrosesan bahasa alami seperti **terjemahan mesin**, **penjawab pertanyaan otomatis**, dan **generasi teks deskriptif untuk video**. Berbeda dengan model RNN tradisional yang memerlukan panjang input-output sama, arsitektur ini memungkinkan fleksibilitas dalam struktur data. Contoh aplikasi nyata: - Terjemahan: "Mary eats apples" (Bahasa Inggris) → "Marie mange des pommes" (Bahasa Prancis) - QA System: "Tim is playing in his room.||Where is Tim?" → "Tim is in his room" - Video captioning: Deretan frame video → "Seorang wanita sedang memasak di dapur"

## Arsitektur Encoder

Bagian encoder terdiri dari beberapa unit recurrent (biasanya RNN atau variannya) yang memproses setiap elemen input secara berurutan. Setiap unit: 1. Menerima satu elemen input (misalnya kata dalam kalimat) 2. Mengumpulkan informasi kontekstual 3. Meneruskan state tersembunyi (hidden state) ke unit berikutnya. Proses ini analog dengan **penerjemah manusia yang mendengarkan seluruh kalimat** sebelum mulai menerjemahkan. Setelah memproses seluruh input, encoder menghasilkan **encoder vector** (state tersembunyi akhir) yang mengkapsulasi informasi esensial dari seluruh urutan input.

## Encoder Vector sebagai Memori Kontekstual

Encoder vector berfungsi sebagai: - Representasi terpadu seluruh input - State awal untuk decoder - "Memori kerja" yang menyimpan dependensi jangka panjang Dalam implementasi teknis, vector ini dihitung melalui fungsi:  $h_{\text{encoder}} = f(W * x_t + U * h_{t-1} + b)$  di mana  $f$  adalah fungsi aktivasi,  $W$  dan  $U$  matriks berat,  $x_t$  input timestep- $t$ , dan  $b$  bias.

## Mekanisme Decoder

Decoder merupakan jaringan recurrent yang menggunakan encoder vector sebagai state awal untuk memulai generasi output. Pada setiap timestep: 1. Menerima state tersembunyi sebelumnya 2. Menghasilkan prediksi output 3. Memperbarui state tersembunyi untuk timestep berikutnya Contoh proses terjemahan: 1. Encoder vector (yang menyimpan makna "Mary eats apples") diinisialisasi sebagai  $h_0$  decoder 2. Decoder menghasilkan "Marie" sebagai output pertama 3. State tersembunyi diperbarui dengan mempertimbangkan output sebelumnya 4. Proses berlanjut hingga menghasilkan tanda akhir kalimat

## Aplikasi dan Variasi Model

Selain terjemahan mesin, arsitektur ini digunakan untuk: - **Text Summarization:** Input dokumen panjang → output ringkasan - **Speech Recognition:** Input sinyal audio → output transkripsi teks - **Image Captioning:** Input citra → deskripsi tekstual Variasi modern meliputi: - **Attention Mechanism:** Memungkinkan decoder fokus pada bagian spesifik encoder vector - **Transformer Architecture:** Menggantikan RNN dengan self-attention untuk penangkapan konteks lebih baik

[!cornell] ##### Summary

**Model encoder-decoder** memungkinkan pemrosesan urutan dengan panjang input-output berbeda melalui dua komponen utama: **encoder** yang mengompresi input menjadi representasi vektor, dan **decoder** yang menggunakan vektor tersebut untuk membangkitkan output berurutan. Arsitektur ini menjadi landasan sistem terjemahan mesin modern dan aplikasi NLP lanjutan, dengan kemampuan menangani dependensi jangka panjang melalui mekanisme state tersembunyi. **Encoder vector** berperan kritis sebagai pembawa informasi kontekstual antar komponen.

[!ad-libitum]- Additional Information

## Evaluasi Kinerja Model

Metrik evaluasi utama untuk model seq2seq: - **BLEU Score:** Mengukur kecocokan n-gram antara output model dan referensi manusia - **ROUGE:** Fokus pada recall n-gram

untuk tugas summarization - **METEOR**: Mempertimbangkan sinonim dan stemming dalam evaluasi

Tantangan utama meliputi masalah **penyusutan kosa kata** (vocabulary bottleneck) dan **generasi output repetitif** yang diatasi dengan teknik seperti beam search dan sampling stokastik.

## Implementasi Praktis dengan TensorFlow

Contoh implementasi dasar encoder-decoder: ```python

### Encoder

---

```
encoder_inputs = Input(shape=(max_input_len,)) encoder_embedding =
Embedding(input_vocab_size, 256)(encoder_inputs) encoder_lstm = LSTM(128,
return_state=True) encoder_outputs, state_h, state_c =
encoder_lstm(encoder_embedding)
```

### Decoder

---

```
decoder_inputs = Input(shape=(max_output_len,)) decoder_embedding =
Embedding(output_vocab_size, 256)(decoder_inputs) decoder_lstm = LSTM(128,
return_sequences=True, return_state=True) decoder_outputs, _, _ =
decoder_lstm(decoder_embedding, initial_state=[state_h, state_c]) decoder_dense =
Dense(output_vocab_size, activation='softmax') output =
decoder_dense(decoder_outputs) ```
```

## Tantangan dalam Training

- **Alignment Problem**: Hubungan kompleks antara posisi input-output
- **Exposure Bias**: Discrepancy antara training (menggunakan ground truth) dan inference (menggunakan prediksi model)
- **Mode Collapse**: Kecenderungan menghasilkan output generik berulang

Solusi terkini meliputi teknik curriculum learning, reinforcement learning, dan adversarial training.

## Self-Exploration Projects

1. Bangun model terjemahan Inggris-Indonesia menggunakan dataset Paralel TED Talks
2. Implementasikan beam search decoding dengan panjang beam 3-5



3. Eksperimen dengan mekanisme attention sederhana berbasis dot-product
4. Ukur pengaruh ukuran embedding (64 vs 256 dimensi) terhadap akurasi BLEU

### Tools dan Resources

- **Framework:** TensorFlow Seq2Seq Tutorial, PyTorch TorchText
- **Dataset:** WMT14, Multi30k, OPUS
- **Library:** HuggingFace Transformers, OpenNMT-py
- **Visualization:** TensorBoard Embedding Projector

### Further Reading

- "Neural Machine Translation by Jointly Learning to Align and Translate" (Bahdanau et al. 2014)
- "Attention Is All You Need" (Vaswani et al. 2017)
- "Sequence to Sequence Learning with Neural Networks" (Sutskever et al. 2014)
- Buku "Natural Language Processing with PyTorch" oleh Delip Rao